SINGLE GRAIN · OPEN CHALLENGE

General 000 Automation Engineer

JDE Work Order Automation • Mohamed AH

github.com/Mohamed-AH/automation

4,448	100%	14 mo	6
PDFS UPLOADED	SUCCESS RATE	PRODUCTION RUN	AUTOMATED STAGES

Repository: github.com/Mohamed-AH/automation

Part 1: Role Definition

The role I want: Automation Engineer.

My ownership would sit at the intersection of operations and engineering — finding the manual, friction-heavy processes that quietly drain hours each week, then building reliable automation pipelines to handle them. Concretely: auditing workflows to surface what's repetitive, designing and shipping the tooling to replace it, and documenting where human judgment still needs to stay in the loop.

First 90 days:

- Days 1–30: Understand the delivery workflows. Map where time gets lost. Talk to the people doing the repetitive work, not just the people managing it.
- Days 31–60: Ship one working automation for the highest-friction process identified. Get it running in production, not just a demo.
- Days 61–90: Document failure modes, set up monitoring, and start on the second target.

Why Single Grain:

Single Grain's model — AI handles the volume, humans focus on the thinking — is exactly what this project demonstrates in practice. I didn't build this automation to replace anyone. I built it so the people involved could stop doing work that a script could do better. That's the same philosophy.

Part 2: Evidence of Excellence

The Project

Our team processes large batches of work orders inside JD Edwards (JDE) — a complex enterprise system where everything is manual by default. Each work order means extracting data from PDFs, creating records in JDE, entering line items, processing inventory, and uploading documentation. Done by hand, a batch like this takes days of repetitive clicking and copy-pasting, with plenty of room for entry errors.

We automated the entire pipeline end-to-end across six stages.

The Six Stages

00 Pre-processing (`00 preprocess` **)** Uses Python and the Claude API to extract structured data from raw work order PDFs — eliminating manual data entry at the source.

01 Work Order Creation (`01 woCreation` **)** Automates the initial generation of work orders inside JDE, navigating the menu hierarchy and populating the required fields programmatically.

02 Parts Details Entry (`02 enterWO` **)** Handles the systematic entry of parts and material requirements into each work order's grid, running through 600+ line items across batches.

03 Work Order Issuance (`03 issue` **)** Executes the final issuance step that moves work orders into active processing, including a manual verification checkpoint for unit-of-measure discrepancies.

04 Work Order PDF Upload (`04 upload` **)** Attaches the original documentation to each work order in JDE using a custom VBScript and AutoIt stack — the most technically involved stage, detailed below.

05 Error Cross-Checking (`05 check` **)** Performs a final automated audit across all processed work orders, flagging parts with unresolved quantities or data mismatches for human review.

Technical Highlights

Challenge 1 — The Upload Problem

The naive approach here is Puppeteer. It's what most AI tools will suggest when you ask how to automate a file upload in a browser.

The problem: modern browsers deliberately block any script from programmatically populating a `<input type="file">` field or triggering the native OS file dialog. It's a security feature, and it's not going away. Puppeteer can work around it using Chrome DevTools Protocol to inject file paths

directly — but that requires running a full Puppeteer session, which means automating the login, storing credentials somewhere the script can read them, and hoping JDE's session handling doesn't get in the way.

We took a different path.

The system uses **VBScript** (`workorder_process.vbs`) to control an already-open Internet Explorer window via COM's `Shell.Application` object. IE exposes its DOM to VBScript through `execScript()` — something no modern browser allows. VBScript navigates the JDE attachment UI, finds the upload dialog trigger, and fires it. Then **AutoIt** (`UploadFile.au3`) takes over: it waits for the native Windows "Choose File to Upload" dialog to appear, injects the full file path into the `Edit1` control, and clicks `Button1` to confirm.

The AutoIt script is 21 lines. It does exactly one thing and does it reliably.

Results from `04 upload/Autoit/upload_log.txt`: - 4,448 unique PDF files uploaded - 4,462 total log entries (including re-runs) - **Date range:** November 26, 2024 → January 22, 2026 (14 months) - **File range:** 610122 – 614698 - **Success rate:** 100% — not a single error entry in the log - **Cadence:** ~36 seconds per file, consistent across the entire run

```
2024/11/26 16:06:28 - File uploaded: D:\auto\upload\pdfs\610122.pdf
...
2026/01/22 19:23:13 - File uploaded: D:\auto\04 upload\pdfs\614698.pdf
```

This wasn't the obvious solution. It was the one that actually worked.

Challenge 2 — Keeping Credentials Out of the Code

The standard AI recommendation for automating a web application is: use Puppeteer, automate the login, store your credentials in an environment variable or config file.

That approach has real problems. Credentials in config files get committed to repos, copied into logs, exposed in error messages. JDE also actively resists automated logins — sessions managed by a bot behave differently than sessions from a real browser, and the system notices.

Our solution: **don't automate the login at all.**

The operator logs in manually through a normal browser session. Once authenticated, the JavaScript automation scripts (`WoCreation.js`, `dragon3.js`, `issues4.js`) are pasted directly into the browser's DevTools console and run inside the live, authenticated session. The automation picks up after the human has already established the session — credentials never appear in any file the code touches.

This keeps the security boundary clean and sidesteps JDE's bot-detection entirely. It also means the human is still in control of the one step that actually requires a human: proving who they are.

Supporting Documentation

Artifact	Location	Purpose
Repository	github.com/Mohamed-AH/automation	Full source code
Upload log	`04 upload/Autoit/upload_log.txt`	Primary evidence — 4,462 timestamped entries
VBScript uploader	`04 upload/workorder_process.vbs`	IE automation + AutoIt delegation
AutoIt handler	`04 upload/Autoit/UploadFile.au3`	Native dialog interaction
PDF extractor	`00 preprocess/claude.py`	Claude API integration
Browser scripts	`01 woCreation/`, `02 enterWO/`, `03 issue/`	Console-paste JS automation
Verification script	`05 check/partsDetailAutomation.js`	Post-process audit

AI tools used: Claude API (anthropic) for PDF data extraction.

Failure modes documented: Missing PDFs logged with expected path; VBScript validates environment variables before execution; browser scripts include retry logic with exponential backoff and infinite-loop protection; `error.txt` captures issue-stage failures for manual review.

Human oversight checkpoints: Login (always manual), unit-of-measure verification (flagged with a red banner in the issue script), final audit review of flagged parts in the check stage.

Part 3: AI Competency

I have a genuine interest in finding the processes that frustrate people most — the ones that eat hours through pure repetition — and figuring out where automation can take over. That's what drove this project, and it's what keeps me paying attention to what's changing in the space.

To stay current I follow the field closely through AI Twitter, where the practical signal tends to show up faster than anywhere else. Right now I'm running a **Hermes agent** on a Google Cloud instance, connected to a Telegram chat interface, to get hands-on time with autonomous agent workflows — understanding where they're actually reliable versus where they still need guardrails.

The most meaningful thing I've built with AI so far is the PDF extraction stage of this project. Before it existed, someone had to open each PDF, read the work order data, and type it into a spreadsheet. The Claude API replaced that entirely — reading the documents, understanding the structure, and outputting clean, validated CSV. Not a prototype. It ran in production for 14 months.

Where I see this going by 2028:

AI won't be a "cool feature" anymore. It'll be infrastructure — the same way electricity or the internet is just always there. The quiet engine in the background handling the high-effort, complex tasks that currently drain our time.

Think about what that unlocks: farm harvests that are meaningfully more efficient, health problems caught long before symptoms appear, bureaucratic processes that currently drag everything down finally cut through.

As the cost of executing these tasks drops, the old model of "working just to survive" stops making sense. Universal income stops being a debate and becomes a practical necessity — not because no one works, but because the economic logic shifts when machines handle the repetitive volume.

That's what I think this is actually about: decoupling our time from our basic needs. The goal isn't a world where no one does anything. It's one where all the tedious, repetitive work is handled, and people are finally free to focus on what only people can do — building communities, making things, and having the time to actually live.